```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
text = """
artificial intelligence is transforming modern society.
it is used in healthcare finance education and transportation.
machine learning allows systems to improve automatically with experience.
data plays a critical role in training intelligent systems.
large datasets help models learn complex patterns.
deep learning uses multi layer neural networks.
neural networks are inspired by biological neurons.
each neuron processes input and produces an output.
training a neural network requires optimization techniques.
gradient descent minimizes the loss function.
"""
```

```
text = text.lower().replace(".", "")

tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])

word_index = tokenizer.word_index
index_word = tokenizer.index_word
vocab_size = len(word_index) + 1
```

```
tokens = tokenizer.texts_to_sequences([text])[0]

SEQ_LENGTH = 5
X = []
y = []

for i in range(len(tokens) - SEQ_LENGTH):
    X.append(tokens[i:i + SEQ_LENGTH])
    y.append(tokens[i + SEQ_LENGTH])

X = np.array(X)
y = np.array(y)
```

```
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=64, input_length=SEQ_LENGTH),
    SimpleRNN(128),
    Dense(vocab_size, activation="softmax")
])

model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="adam"
)

model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dense_3 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)

```
model.fit(
    X,
    y,
    epochs=40,
    batch_size=16
```

```
)
```

```
Epoch 1/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 4.1994
Epoch 2/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 4.0928
Epoch 3/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 3.9962
Epoch 4/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 3.8905
Epoch 5/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 3.7795
Epoch 6/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 3.6103
Epoch 7/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 3.4003
Epoch 8/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 3.1186
Epoch 9/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 2.8155
Epoch 10/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 2.4053
Epoch 11/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 1.9563
Epoch 12/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 1.5102
Epoch 13/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 1.1646
Epoch 14/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.9078
Epoch 15/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 0.7190
Epoch 16/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.5623
Epoch 17/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.4246
Epoch 18/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.3292
Epoch 19/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.2833
Epoch 20/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.2384
Epoch 21/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.1880
Epoch 22/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.1654
Epoch 23/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.1420
Epoch 24/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.1282
Epoch 25/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.1210
Epoch 26/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 0.0982
Epoch 27/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.0900
Epoch 28/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 0.0880
Epoch 29/40
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - loss: 0.0775
```

```
def generate_text(seed_text, length=30):
    seed_text = seed_text.lower().split()
    generated = seed_text.copy()

    for _ in range(length):
        encoded = tokenizer.texts_to_sequences(
            [" ".join(generated[-SEQ_LENGTH:])]
        )[0]

        encoded = pad_sequences(
            [encoded], maxlen=SEQ_LENGTH, padding="pre"
        )

        prediction = model.predict(encoded, verbose=0)
        next_word_index = np.argmax(prediction)
        next_word = index_word.get(next_word_index, "")

        generated.append(next_word)

    return " ".join(generated)
```

```
print(generate_text("artificial intelligence is"))
```

```
artificial intelligence is training data society it is help in healthcare finance education and transportation machine learr
```